

POR QUE PROJETOS CONTINUAM FALHANDO?

Uma análise crítica do modelo tradicional de desenvolvimento de software e os dados que revelam uma crise sistêmica na entrega de valor.

<30%

TAXA DE SUCESSO COMPLETO

Projetos preditivos (Waterfall) historicamente entregam sucesso completo em menos de 30% dos casos, segundo o Chaos Report do Standish Group.

>45%

FUNCIONALIDADES NUNCA USADAS

Mais de 45% das funcionalidades desenvolvidas em escopos fechados nunca ou raramente são utilizadas pelo cliente final – esforço desperdiçado em escala industrial.



O PARADIGMA DO MODELO EM CASCATA (WATERFALL)

O modelo Waterfall organiza o desenvolvimento em fases sequenciais rígidas: **Análise de Requisitos** → **Design** → **Desenvolvimento** → **Testes** → **Implantação**. Cada fase precisa ser concluída antes que a próxima comece — e é exatamente aí que residem suas falhas estruturais.

1

PREMISSA FALSA 1

"O cliente sabe exatamente o que quer no Dia 1 do projeto." Na prática, requisitos evoluem com o aprendizado. O que parece claro no início raramente sobrevive ao contato com a realidade do desenvolvimento.

2

PREMISSA FALSA 2

"Nada mudará no mercado ou na tecnologia durante os próximos 6 meses." Mercados mudam, concorrentes lançam produtos, regulações surgem — o mundo não espera o projeto terminar.

3

PREMISSA FALSA 3

"Software é como a construção civil: planejar tudo antes garante o resultado." Diferente de uma obra física, software é conhecimento puro — intangível, mutável e profundamente dependente de descobertas ao longo do caminho.

O RISCO ACUMULADO DO WATERFALL

O maior perigo do modelo tradicional não é o planejamento em si — é o momento em que o erro é descoberto. No Waterfall, o risco se acumula silenciosamente ao longo de meses, explodindo apenas no momento do deploy.



⊗ O impacto crítico: quanto mais tarde o erro é descoberto, mais caro e devastador é corrigi-lo. No Waterfall, o produto errado só é revelado no final — quando já não há tempo nem orçamento para mudar.



BLOCO 2

O FRAMEWORK CYNEFIN E A COMPLEXIDADE

Antes de escolher qualquer ferramenta de gestão, é preciso entender **em qual tipo de terreno o problema reside**. O modelo Cynefin, criado por Dave Snowden, oferece uma taxonomia poderosa para classificar problemas e escolher a abordagem correta – evitando o erro fatal de aplicar soluções simples a problemas complexos.

A pergunta fundamental que toda equipe deve fazer antes de iniciar um projeto: *"Qual é a natureza do problema que estamos tentando resolver?"*

SISTEMAS COMPLEXOS VS. COMPLICADOS

SIMPLES (REGRAS CLARAS)

Causa e efeito são evidentes. Aplicam-se melhores práticas padronizadas.

Exemplos: Trocar uma lâmpada, formatar um PC.

COMPLICADO (ANÁLISE DE ESPECIALISTAS)

Causa e efeito separados no tempo. Exige análise profunda e boas práticas.

Exemplos: Montar um motor, migrar banco de dados.

COMPLEXO (EMERGENTE)

Causa e efeito só vistos a posteriori. Exige experimentação rápida – Empirismo.

Exemplos: Criar uma nova startup, desenvolver um App IoT.

CAÓTICO (RESPOSTA DE EMERGÊNCIA)

Sem relação perceptível de causa e efeito. Ação imediata para conter danos.

Exemplos: Data center pegando fogo, invasão de sistema.

📄 **A Conclusão para o Time:** Software, inovação e arquitetura de nuvem habitam o domínio **Complexo**. No domínio complexo, não adianta "analisar mais tempo" – a única forma de progredir é através do **Empirismo: Experimentar → Perceber → Responder**.

A MUDANÇA DE PARADIGMA: ESCOPO FECHADO VS. VALOR ENTREGUE

O modelo tradicional e o modelo ágil divergem fundamentalmente em **o que é fixo e o que é variável**. Essa inversão não é apenas técnica — é uma mudança filosófica sobre onde reside o risco e como o valor é gerado.

TRIÂNGULO DE FERRO TRADICIONAL (RÍGIDO)

Escopo Fixo → Custo e Tempo Variáveis

O escopo é definido no início e não pode mudar. Como consequência, custo e prazo estouram para acomodar a realidade — gerando projetos atrasados, acima do orçamento e frequentemente entregando o produto errado.

TRIÂNGULO DE FERRO ÁGIL (INVERTIDO)

Custo e Tempo Fixos (Sprints) → Escopo Variável

O tempo e o custo são fixados em ciclos curtos (Sprints). O escopo é ajustado continuamente pelo valor real percebido — garantindo que cada entrega seja relevante e que o aprendizado guie as próximas decisões.

O MANIFESTO ÁGIL SEM DOGMAS

Os 4 Valores do Manifesto Ágil não são regras religiosas – são princípios práticos para quem constrói sistemas reais. Cada valor carrega um significado concreto que vai muito além do modismo corporativo.

→ **INDIVÍDUOS E INTERAÇÕES MAIS QUE PROCESSOS E FERRAMENTAS**

Ferramentas no Jira não resolvem a falta de conversa entre o desenvolvedor e o Product Owner. O processo serve às pessoas – não o contrário.

→ **COLABORAÇÃO COM O CLIENTE MAIS QUE NEGOCIAÇÃO DE CONTRATOS**

O contrato rígido não impede que o negócio vá à falência se o produto for inútil. A parceria real supera qualquer cláusula contratual.

→ **SOFTWARE EM FUNCIONAMENTO MAIS QUE DOCUMENTAÇÃO ABRANGENTE**

Um documento Word de 200 páginas não gera valor se a API estiver fora do ar. Documentação é meio, não fim.

→ **RESPONDER A MUDANÇAS MAIS QUE SEGUIR UM PLANO**

Ter a coragem de mudar de rota quando os dados mostram que o usuário quer outra coisa. Planos são hipóteses – dados são realidade.

DINÂMICA PRÁTICA DE ESTIMATIVAS

Esta dinâmica vai provar na prática como o ser humano é péssimo em estimar horas absolutas — e como a estimativa relativa funciona melhor. O cenário do exercício é "**A Reforma da Casa**", com 6 tarefas de complexidades propositalmente diferentes.

AS 6 TAREFAS DO EXERCÍCIO

1. Pintar uma parede de 3x3m com tinta branca
2. Trocar uma lâmpada do teto
3. Instalar um vaso sanitário novo (incluindo encanamento)
4. Pintar um afresco detalhado no teto da sala de estar
5. Trocar o piso de cerâmica de toda a cozinha (20m²)
6. Trocar a tomada da parede

ESTRUTURA DA DINÂMICA

Θ1

DIVISÃO DE GRUPOS

4 a 5 integrantes por mesa ou sala virtual.

Θ2

RODADA 1

Estimativa Absoluta em Horas (Tabela Rígida).

Θ3

RODADA 2

Estimativa Relativa com Planning Poker (Fibonacci).

Θ4

RODADA 3

Abordagem NoEstimates (Contagem de Itens / Sizing).

AS TRÊS RODADAS: DA HORA AO FLUXO

RODADA 1: ESTIMATIVA RÍGIDA EM HORAS

Cada grupo fornece o número exato de horas para cada tarefa. **Resultado esperado:** números completamente discrepantes — ex: vaso sanitário variando de 2h a 12h entre grupos. **Discussão:** Por que as horas variaram tanto? Porque a experiência prévia e as premissas não ditas de cada pessoa são diferentes.

RODADA 2: PLANNING POKER / ESTIMATIVA RELATIVA

Usa a Sequência de Fibonacci Adaptada (1, 2, 3, 5, 8, 13, 20, 40). Define-se uma **Tarefa Âncora (Tamanho 1):** "Trocar a tomada da parede". Os grupos estimam as demais tarefas comparando complexidade relativa. **Resultado esperado:** alinhamento rápido — pintar o afresco é claramente um 20 ou 40, enquanto trocar a lâmpada é 1.

RODADA 3: NOESTIMATES / RIGHT-SIZING

A provocação: E se, em vez de gastar horas discutindo se um item é 3 ou 5, nós simplesmente quebrássemos todas as tarefas em itens pequenos de tamanho similar (1 a 3) e apenas contássemos a quantidade de itens entregues? O foco muda de estimativa para vazão real.

DEBRIEFING: AS TRÊS CONCLUSÕES DE LIDERANÇA

A dinâmica das estimativas não é apenas um exercício lúdico – ela revela verdades fundamentais sobre como times de engenharia devem pensar sobre planejamento e previsibilidade.

1 HORAS ABSOLUTAS CRIAM FALSAS EXPECTATIVAS

Estimar em horas no início de projetos complexos gera compromissos ilusórios. O número parece preciso, mas esconde uma variância enorme – e essa ilusão de precisão é mais perigosa do que a incerteza assumida.

2 A ESTIMATIVA RELATIVA MEDE COMPLEXIDADE E RISCO

O Planning Poker não serve apenas para obter um número – seu verdadeiro valor é **gerar conversa no time e revelar divergências de entendimento**. Quando um desenvolvedor diz 3 e outro diz 13, há uma conversa crítica que precisa acontecer.

3 FOCO NA VAZÃO REAL

O que prediz o futuro de um time não é a estimativa no papel, mas o **histórico do seu fluxo de entregas**. Times maduros medem throughput, cycle time e lead time – não horas estimadas versus realizadas.